Описание проекта

Тестируемые приложения:

1. Публичный неофициальный программный интерфейс **Kinopoisk API Unofficial** (доступный по адресу https://kinopoiskapiunofficial.tech) и WEB сайт: kinopoisk.ru.

   Основное назначение приложения: Предоставление сторонним разработчикам структурированных данных о фильмах, сериалах, актерском составе, кассовых сборах, премиях и связанных медиа-материалах базы данных «Кинопоиск» посредством REST API шлюза. Сервис служит бэкенд-платформой для интеграции кинематографического контента в сторонние веб-сайты, мобильные и аналитические приложения

2. **WEB сайта kinopoisk.ru**

   Основное назначение : предоставление пользователям максимально полной информации о кинематографе (фильмах, сериалах, мультфильмах), а также обеспечение доступа к их легальному онлайн-просмотру.

   Проверяемую функциональность можно разделить на два ключевых блока:

1. Информационно-энциклопедическая база

- **Поиск и картотека:** Сайт собирает детальные данные о кинокартинах (актерский состав, режиссеры, трейлеры, сюжет, кассовые сборы, награды) и деятелях киноиндустрии.
- **Рейтинги:** Формирование авторитетных рейтингов (например, знаменитый «Топ-250 лучших фильмов»), основанных на оценках миллионов пользователей.
- **Новости и медиа:** Публикация рецензий, критических статей, новостей из мира кино, интервью и репортажей с фестивалей.

Онлайн-кинотеатр и стриминг

- **Просмотр контента:** Интеграция с медиаплатформой «Плюс Студия» (Яндекс) позволяет смотреть тысячи фильмов и сериалов по подписке прямо на сайте или через приложения для Smart TV и смартфонов.

Объект тестирования

Объектом UX тестирования выбрана проверка качественных характеристик системы.

Объектом UI тестирования выбраны три функции: глобальный поиск контента по фильтру, авторизация и личный профиль, Онлайн-кинотеатр.

Объектом тестирования API выбранная ключевая функция: Модуль программного взаимодействия с каталогом фильмов, инкапсулированный в компонентах.

Подход к тестированию

Для верификации разработанного программного обеспечения планируется проведение следующих видов тестирования:

- Проведение сквозного автоматизированного тестирования графического интерфейса пользователя веб-ресурса https://kinopoisk.ru;
- Проведение пользовательского тестирования на предмет проверки качественных характеристик системы;
- Интеграционное тестирование API (основной вид): Контроль транспортного протокола HTTP, валидация статус-кодов (200 OK, 404 Not Found);
- Функциональное тестирование на уровне данных: Десериализация JSON-ответов сервера и проверка бизнес-логики: соответствие текстовых полей эталонам (например, проверка поля nameRu), подтверждение структуры объектов (наличие ключа films в массивах топов);
- Негативное тестирование: Моделирование некорректных запросов к API со стороны клиента для проверки отказоустойчивости системы и корректности обработки ошибок интеграционным шлюзом.

Расписание

№	Этап проекта / Задача	Сроки выполнения
	Тестирование UI	
	Этап 1. Подготовка и проектирование (Дни 1–2)	
1	**Анализ** UI-компонентов сайта kinopoisk.ru (исследование DOM-дерева, определение стабильных локаторов для поиска, карточек фильмов и фильтров).	Неделя 1 (день 1)
2	**Развертывание** локального окружения, настройка изоляции сессии Chrome через профиль	Неделя 1 (день 2)
3	**Проектирование** архитектуры фреймворка. Создание базового класса BasePage с механизмами динамических ожиданий (WebDriverWait).**Создание** структуры конфигурационных файлов (requirements.txt, .env, .gitignore).	Неделя 1 (день 2)
	Этап 2. Разработка слоев Page Object и тестов поиска (Дни 3–5)	
4	**Реализация** классов MainPage и SearchResultsPage.**Написание** и **отладка** Тест-кейсов №1, №2, №3 (Точный поиск фильма, поиск актера, проверка алгоритма исправления раскладки "Bynthcntkkfh").	Неделя 1 (день 3)
5	**Реализация** сложного класса фильтрации Top250Page (логика работы с выпадающими списками стран, жанров и годов).**Написание** и **отладка** Тест-кейса №4 (Комбинаторный поиск фильма «Подольские курсанты» и валидация выдачи по стране «Россия»).	Неделя 1 (дни 4-5)
6	**Промежуточное регрессионное тестирование** разработанных скриптов. Оптимизация локаторов под динамические изменения верстки (борьба с Flaky-тестами).	Неделя 1 (день 5)
	Этап 3. Автоматизация защищенных зон и API (Дни 6–7)	
7	**Реализация** классов MoviePage и AuthProfilesPage.**Написание** и **отладка** Тест-кейса №5 (Попытка добавления в «Буду смотреть» без авторизации) и Тест-кейса №6 (Блокировка просмотра платного плеера). Обход защиты с помощью проверки редиректа на passport.yandex.ru.	Неделя 1 (дни 6-7)
8	**Развертывание слоя** интеграционных API-тестов. Настройка глобальных метаданных и группировки модулей через pytestmark.**Безопасное вынесение** API_KEY в файлы среды .env.	Неделя 1 (день 7)
	Этап 4. Внедрение отчетности Allure и стабилизация (Дни 8–9)	
9	**Интеграция Allure Report** во фреймворк. Разметка тестовых сценариев шагами (with allure.step), добавление уровней критичности (severity) и метаданных.**Реализация перехвата ошибок** в tearDown: написание кода для автоматического снятия и прикрепления скриншотов в Allure при падении тестов.	Неделя 2 (день 8)
	Этап 5. Финализация и аналитика (День 9)	

№	Этап проекта / Задача	Сроки выполнения
10	**Итоговый прогон всего тестового сьюта**. Формирование финального интерактивного HTML-отчета в Allure.**Документирование:** Оформление пояснительной записки.	Неделя 2 (день 9)
	Тестирование UX (день 10)	
1	Написание чек листа проверки качественных характеристик системы, проведение тестирования.	Неделя 2 (день 10)
	Тестирование API	
1	**Аналитика:** Изучение документации Kinopoisk API, проектирование архитектуры тест-кейсов.	Неделя 2 (дни 11–12)
2	**Разработка:** Реализация HTTP-клиента и компонентов на Python (kinopoisk_api.py).	Неделя 2 (дни 13–14)
3	**Автоматизация:** Написание скриптов автотестов в фреймворке Pytest (test_films.py).	Неделя 3 (дни 15-16)
4	**Внедрение отчетности:** Интеграция Allure Framework, настройка файлов метаданных.	Неделя 3 (день 16)
5	**Тестовое выполнение:** Локальные запуски, отладка стабильности, фиксация результатов.	Неделя 3 (день 16)
6	**Документирование:** Оформление пояснительной записки.	Неделя 3 (день 17)

Необходимые ресурсы

- Браузеры: Google Chrome (актуальная версия), Яндекс Браузер.
- Устройства: Настольный ПК (Windows 10).

Аппаратные ресурсы:

- Персональный компьютер (рабочая станция тестировщика): CPU 2.0 ГГц, RAM 8 ГБ, SSD-накопитель.
- Стабильный выделенный канал связи Интернет (скорость не менее 10 Мбит/с) для обращения к внешнему облачному шлюзу.

Программные инструменты и среды:

• Язык программирования: Python версии 3.10+.

• Тестовый фреймворк: Pytest.

• Библиотека сетевого транспорта: Requests.

• Инструмент построения отчетов: Allure Commandline (требует установленной Java Runtime Environment JRE 8+).

• Среда разработки (IDE): Visual Studio Code с расширениями Python и Pylance для форматирования кода по стандарту PEP 8.

• Инструмент предварительного ручного исследования: Postman (используется для предварительной разведки API, проверки структуры JSON-ответов и валидации авторизационных токенов перед их программированием в коде).

• Система управления тестированием (TMS): TestIT (или встроенная структура Allure) .

Итоговый отчет о тестировании UI-тестов и API-тестов

Отчет о тестировании UI-тестов

Описание реализации программного модуля сквозного автоматизированного тестирования веб-ресурса «Кинопоиск»

1. Конфигурация тестового окружения и архитектура веб-драйвера
Для проведения сквозного автоматизированного тестирования графического интерфейса пользователя веб-ресурса https://kinopoisk.ru был разработан специализированный программный модуль на базе фреймворка **Unittest** и инструментария **Selenium WebDriver**.

В ходе проектирования инициализирующего метода (setUp) были решены ключевые проблемы стабильности и отказоустойчивости веб-драйвера при работе со сложными SPA-приложениями (Single Page Applications), защищенными алгоритмами обнаружения ботов:

Персистентность пользовательских сессий: Организовано подключение к выделенной локальной директории хранения профиля браузера selenium*chrome*profile. Данное решение позволяет сохранять и повторно использовать кэш, куки (Cookies) и авторизационные токены экосистемы Яндекса между тестовыми сессиями. Это исключает необходимость повторного прохождения систем капчи (например, Yandex SmartCaptcha) или двухфакторной аутентификации при каждом изолированном запуске теста.

Маскировка признаков автоматизации (AutomationControlled): За счет программного отключения флага AutomationControlled в опциях Blink-движка веб-драйвера минимизируется вероятность идентификации сессии как автоматизированной со стороны систем Web Application Firewall (WAF).

Двухуровневая стратегия синхронизации:

Глобальные неявные ожидания (implicitly_wait(60)) — определяют максимальный лимит времени на поиск базовых узлов в DOM-дереве.

Явные динамические ожидания (WebDriverWait до 40 секунд) — используются для асинхронных React-компонентов в связке с классом expected_conditions (EC), гарантируя появление элементов на веб-странице до начала взаимодействия с ними.

2. Анализ и спецификация реализованных тестовых сценариев
Разработанный тестовый комплекс KinopoiskUITests покрывает три критически важные функциональные зоны веб-ресурса: глобальный поисковый движок, персонализацию профиля (интеграция с личным кабинетом) и разграничение прав доступа в модуле онлайн-кинотеатра.

Функция 1: Глобальный поиск и многофакторная фильтрация контента

- **test*search*movie*by*name (Поиск фильма по точному названию)**: Проверяет базовую цепочку навигации. Скрипт осуществляет поиск интерактивного поля по CSS-селектору, передает строковый литерал «Интерстеллар» и инициирует отправку формы через нажатие клавиши Keys.RETURN. Тест подтверждает корректность выдачи, проверяя вхождение подстроки в текст первого элемента с использованием уникального дата-атрибута (data-tid).
- **test*search*actor*by*name (Поиск по имени актера)**: Проверяет поисковый алгоритм обработки сущностей типа «Персона». Логика теста контролирует автоматическое перенаправление пользователя на персональную карточку актера, верифицируя заголовок страницы с помощью точного XPath-выражения //span[text()='Леонардо ДиКаприо'].
- **test*search*wrong*layout*correction (Автоматическое исправление неверной раскладки)**: Проверяет работу интеллектуального модуля исправления ввода (инверсии раскладки). Скрипт имитирует ошибочный ввод пользователя («Bynthcntkkfh» вместо «Интерстеллар»). Проверка считается успешной, если динамический блок поисковых подсказок (Suggest Block) распознал намерение и вывел корректный текст.
- **test*search*for*the*best (Многофакторный поиск по спискам и фильтрам)**: Наиболее сложный UI-сценарий, проверяющий асинхронную фильтрацию в разделе «250 лучших фильмов». Ввиду высокой динамичности интерфейса, клики по элементам каталога и раскрывающимся кнопкам («Страны», «Жанры», «Годы») дублируются прямым вызовом JavaScript-инструкций через execute*script(), а элементы предварительно скроллятся в область видимости (scrollIntoView).Валидация выполняется на двух уровнях: во-первых, проверяется присутствие и фактическая видимость (is*displayed()) на экране целевого фильма «Подольские курсанты»; во-вторых, скрипт собирает массив всех отфильтрованных плиток фильмов и в цикле проверяет, что текстовое описание каждого элемента содержит маркер выбранной страны («Россия»), отсекая строки актерского состава.

Функция 2: Авторизация и личный профиль
test*add*movie*to*watchlist (Добавление фильма в список «Буду смотреть»): Сценарий валидирует состояние персонализированного интерфейса. Скрипт осуществляет переход на страницу конкретной кинокартины и имитирует нажатие на кнопку добавления в закладки. Для

верификации используется динамический опрос состояния DOM-элемента: тест ожидает изменения HTML-атрибута aria-pressed в значение true, что подтверждает фиксацию действия на стороне бэкенд-сервера пользователя.

Функция 3: Онлайн-кинотеатр

test*play*premium*content*without_subscription (Попытка просмотра платного контента без подписки): Негативный сценарий безопасности, проверяющий бизнес-логику ограничения неавторизованного или неплатёжеспособного доступа к медиа-контенту онлайн-кинотеатра. Скрипт осуществляет переход на страницу платного медиа-ресурса и инициирует клик по кнопке просмотра (селектор data-tid='6cb8d12f').*Проверка включает комплексный анализ:* верифицируется факт автоматического прерывания сессии просмотра и перенаправления (редиректа) пользователя на страницу выбора профилей hd.kinopoisk.ru/profiles либо на внешнюю страницу авторизации Паспорта (passport.yandex, id.ru), защищая платный контент от несанкционированного стриминга.

Тест-кейсы и чек-лист в XLS

UITest IT.xlsx 18540

Результаты проведенного тестирования

Тест-ран выполнения тестов

Отчёт из allure-results

Отчет выполнения UI тест кейсов из allure-results

Ссылку на публичный проект автотестов в GitHub

Аналитическая справка по результатам прогона:

Всего запланировано тест-кейсов: 6

Успешно выполнено (Passed): 6 (100%)

Зафиксировано дефектов (Failed/Broken): 0

В результате проведения автоматизированного UI-тестирования средствами Python + Selenium WebDriver были успешно подтверждены заявленные в дипломе функциональные возможности исследуемой системы. Изоляция сессионных данных с помощью локального профиля браузера Chrome позволила исключить падения тестов, вызванные защитными механизмами авторизации (капча, антифрод) и сквозными редиректами транзитных страниц. Все выбранные ключевые функции веб-приложения покрыты автотестами, дефектов критического уровня («Blocker», «Critical») в проверяемых модулях на момент проведения верификации не обнаружено

Отчет о тестировании публичного неофициального API Кинопоиска v1 / v2.2

1. Конфигурация тестового окружения и архитектура веб-драйвера

1.1. Конфигурация тестового окружения (Test Environment)

Для запуска и выполнения разработанного тестового пакета развернуто окружение со следующими характеристиками:

- Язык программирования: Python 3.10+ (с поддержкой современной типизации данных).
- Тестовый фреймворк: pytest (используется модульный scope для фикстур).
- Библиотека HTTP-запросов: requests — осуществляет прямое синхронное взаимодействие с REST API. Менеджер переменных окружения: python-dotenv (безопасное чтение KINOPOISK*API*KEY из изолированного .env файла).
- Репортер: allure-pytest — перехватывает шаги выполнения (allure.step) и формирует структурированные JSON/арт-отчеты.

1.2. Архитектура веб-драйвера (Web Driver Architecture)

В данном компоненте тестов компонент WebDriver (Selenium) исключен из цепочки вызовов.

Обоснование: Тестирование ведется на сетевом уровне (REST API), что изолирует бизнес-логику от UI-зависимостей.

Сравнение архитектурных подходов в проекте:

UI-тесты: Используют классическую архитектуру W3C WebDriver Client ➔ Browser Driver (e.g., ChromeDriver) ➔ Browser. Она незаменима для сквозных эмуляций действий пользователя на фронтенде. API-тесты: Запросы отправляются напрямую из кода клиента KinopoiskAPI на веб-сервер https://kinopoisk.ru. Это увеличивает скорость выполнения тестов в десятки раз, снижает нестабильность (flakiness) и исключает необходимость управления бинарными файлами драйверов.

2. Функции, подлежащие тестированию (Features Under Test)

В рамках данного тестового набора проверке подвергаются следующие бизнес-функции и эндпоинты цифровой платформы:

1.Каталогизация и поиск фильмов: Возможность получения точных карточек контента по уникальным идентификаторам и ключевым словам.

2.Пагинация и лимиты выборок: Корректность отдачи списков (ограничение до 20 элементов на страницу для коллекций).

3.Финансовый и наградной учет: Обработка вложенных массивов данных (сборы Box Office, кинопремии и номинации).

4.Справочники метаданных: Валидация консистентности системных справочников (сопоставление ID стран и жанров их текстовым эквивалентам).

3. Анализ и спецификация реализованных тестовых сценариев

Все сценарии категоризированы с помощью Allure-аннотаций (Epic, Story, Title), снабжены валидацией статус-кодов и проверкой структуры JSON-ответов.

Название теста	Проверяемые параметры и Assertions
test*get*film_details	Статус-код 200. Соответствие поля nameRu значению "Артур, ты король". Прикрепление JSON-логирования.
test*negativ*get*film*details	Статус-код 404 при передаче несуществующего film_id.
test*get*top_films	Статус-код 200. Наличие корневого ключа списка "films" в структуре ответа.
test*film*search*by*keyword	Статус-код 200. Наличие непустого массива "films". Проверка соответствия очищенного keyword значению "Власть пса". Проверка наличия "pagesCount".
test*get*box_office	Статус-код 200. Проверка обязательных ключей верхнего уровня (total, items). Динамическая валидация полей type и amount у вложенных объектов (при их наличии).
test*films*awards	Статус-код 200. Валидация первого элемента массива премий на строгое соответствие: name "Золотой глобус" и year 2022.
test*get*collections	Статус-код 200. Проверка выполнения бизнес-требования пагинации: количество записей на одной странице 0 < count <= 20.
test*gef*filters	Статус-код 200. Проверка маппинга справочников: genres[id=3] -> "криминал", countries[id=34] -> "Россия".

Выводы по результатам анализа тест-кода:
Покрытие: Код демонстрирует высокий уровень тестового покрытия критических интеграционных точек API.

Изоляция данных: Использование фикстуры api со scope="module" оптимизирует сессии подключения.

Отказоустойчивость: Внедрены защитные проверки (например, if len(data["items"]) > 0), предотвращающие падение тестов по ошибке IndexError при изменении данных на бэкенде

Тест-кейсы и коллекция Postman в JSON

APITest IT.xlsx 14941

kinopoiskapi.postmancollection.json 29584

Результаты проведенного тестирования

Тест-ран выполнения тестов в Postman

Тест-ран выполнения тестов в VS Code

Отчет выполнения API тест кейсов из allure-results

Репозитарий

Аналитическая справка по результатам прогона:

Всего запланировано тест-кейсов: 11

Успешно выполнено (Passed): 11 (100%)

Зафиксировано дефектов (Failed/Broken): 0

В ходе выполнения дипломной работы был успешно спроектирован, реализован и отлажен программный модуль автоматизированного тестирования открытого интеграционного интерфейса API «Кинопоиск».

На основании проделанной работы можно сделать следующие выводы:

1. Эффективность архитектурных решений: Применение паттерна Component Object Model (COM) при проектировании структуры API-клиента (KinopoiskAPI) позволило достичь высокой степени абстракции. Логическое разделение системы на базовый HTTP-транспорт (BaseClient), изолированные компоненты предметной области (FilmsComponent, StaffComponent, MediaComponent, Films*one*Component) существенно снизило связность кода. Это гарантирует масштабируемость автотестов при возможном изменении спецификации эндпоинтов со стороны Кинопоиска.
2. Покрытие функциональных требований: разработанный тестовый набор на базе фреймворка Pytest покрывает ключевые интеграционные сценарии. Реализация как позитивных сценариев (проверка валидности структуры JSON, верификация данных, таких как фильм «Артур, ты король»), так и негативных сценариев (обработка ошибочных запросов с возвратом HTTP-кода 404) позволила комплексно оценить стабильность интеграционного шлюза и корректность обработки исключений серверной частью приложения.
3. Качество отчетности и наблюдаемость системы: За счет глубокой интеграции инструментов Allure Framework была решена задача прозрачности результатов тестирования для смежных участников команды разработки. Использование контекстных шагов (allure.step) и механизмов вложения сырых данных (allure.attach) позволило декомпозировать процесс выполнения тестов до атомарных операций.
4. Успешная автоматизация метаданных отчета: В ходе работы были успешно устранены проблемы пустых аналитических блоков в стандартной конфигурации Allure. Путем внедрения скриптовой инициализации файлов (environment.properties, categories.json, executor.json) непосредственно на этапе инициализации тестовых модулей Python, была достигнута стопроцентная информативность разделов Environment, Categories и Executors. В отчет динамически передаются параметры операционной системы, версии интерпретатора, профили аппаратных требований и правила категоризации дефектов (разделение на баги продукта и дефекты инфраструктуры сети). Таким образом, разработанный комплекс автотестов полностью решает поставленные задачи автоматизации, минимизирует

трудозатраты на проведение регрессионного тестирования, снижает влияние человеческого фактора и готов к бесшовной интеграции в современные конвейеры непрерывной интеграции (CI/CD).

Выводы и рекомендации

Основная проблема автоматизации UI-уровня — высокая динамичность веб-верстки. Прямое использование локаторов (Xpath, CSS-селекторов) внутри тестовых сценариев приводит к деградации тестовой модели при малейшем изменении дизайна сайта.

Для решения этой проблемы в дипломе применен паттерн **Page Object Model (POM)**.

- Каждая веб-страница (или ее крупный логический компонент) представляется в виде отдельного класса.
- Свойства класса инкапсулируют локаторы элементов (селекторы).
- Методы класса описывают действия, которые пользователь может совершить на этой странице (ввод текста, клик, получение состояния).
- Тестовый сценарий оперирует исключительно методами бизнес-логики, не имея прямого доступа к HTML-разметке.

Современные сайты имеют встроенные антифрод-системы, которые мгновенно вычисляют роботов по специальному флагу в коде браузера и препятствуют проведению несанкционированного сбора информации и роботизированных UI-сценариев, возвращая капчу или блокируя IP-адреса стендов тестирования.
Качество платформы и приложений “Кинопоиск” можно оценить как высокое, если рассматривать продукт с точки зрения архитектуры, UX/UI.

Продуктовые и UX/UI достоинства:

- Интуитивный интерфейс: Удобный плеер, лаконичный дизайн карточек фильмов и простая навигация.
- Гибридный формат: Успешное объединение медиа-энциклопедии и современного онлайн-кинотеатра в одном приложении.

На мой взгляд основными недостатками являются:

- Агрессивный антифрод и ограничения для разработчиков: Официальное API закрыто для сторонних разработчиков, а неофициальные методы часто не соответствуют.
- Лицензионные ограничения контента: Из-за ухода зарубежных мейджоров часть каталога недоступна, а старые карточки фильмов (особенно в энциклопедии) иногда путают пользователя: фильм в базе есть, но посмотреть его в онлайн-кинотеатре нельзя.